

SRE and DevOps

Gestão de Infraestruturas de Computação

deti universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

João Paulo Barraca

How Devs differ from Ops

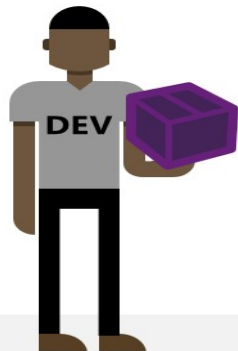
- Development (Dev): Their primary goal is change.
 - They are tasked with writing new code, delivering new features, fixing bugs, and pushing updates to users as quickly as possible.
- Operations (Ops): Their primary goal is stability.
 - They are responsible for keeping the servers running, ensuring security, and maintaining high availability.
 - To Ops, change is the biggest threat to system stability.

How Devs differ from Ops

- **Dev stereotype**

- Impatient
- Temperamental
- Careless about changes
- Prima donna artists
- Ignorant about the “real world”

“It works on my machine”



- **Ops stereotype**

- Slow to act
- Temperamental
- Too rigid about changes
- Always says “No!”
- Too dumb to understand apps

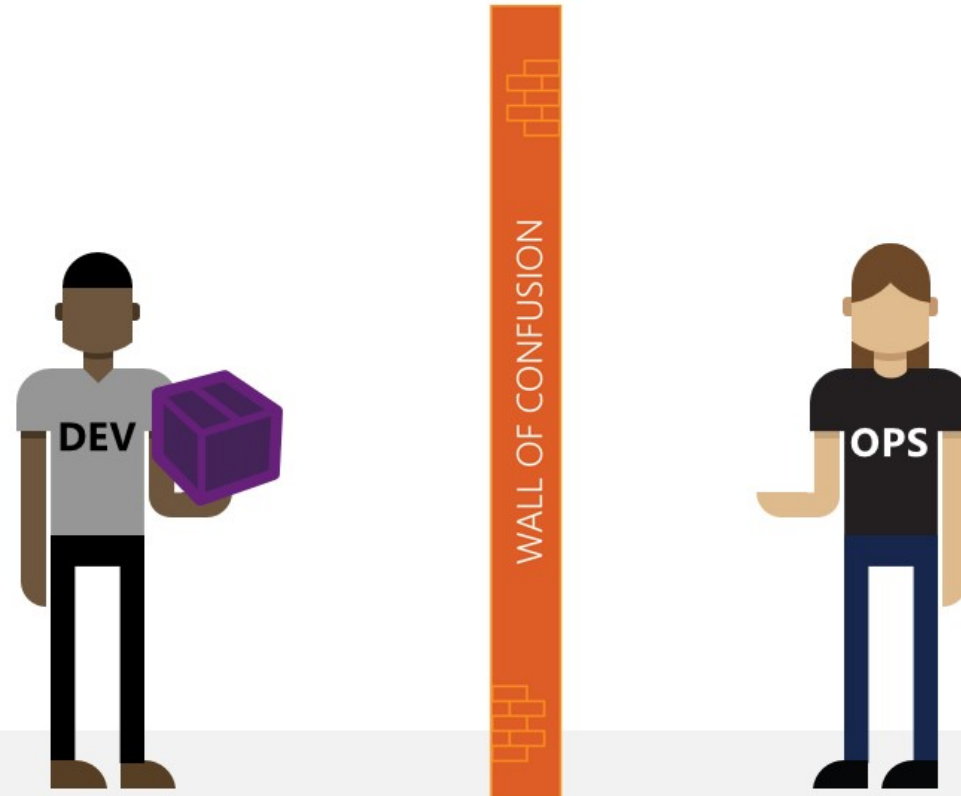
- “Its not the server, it’s your code”



The Wall of Confusion

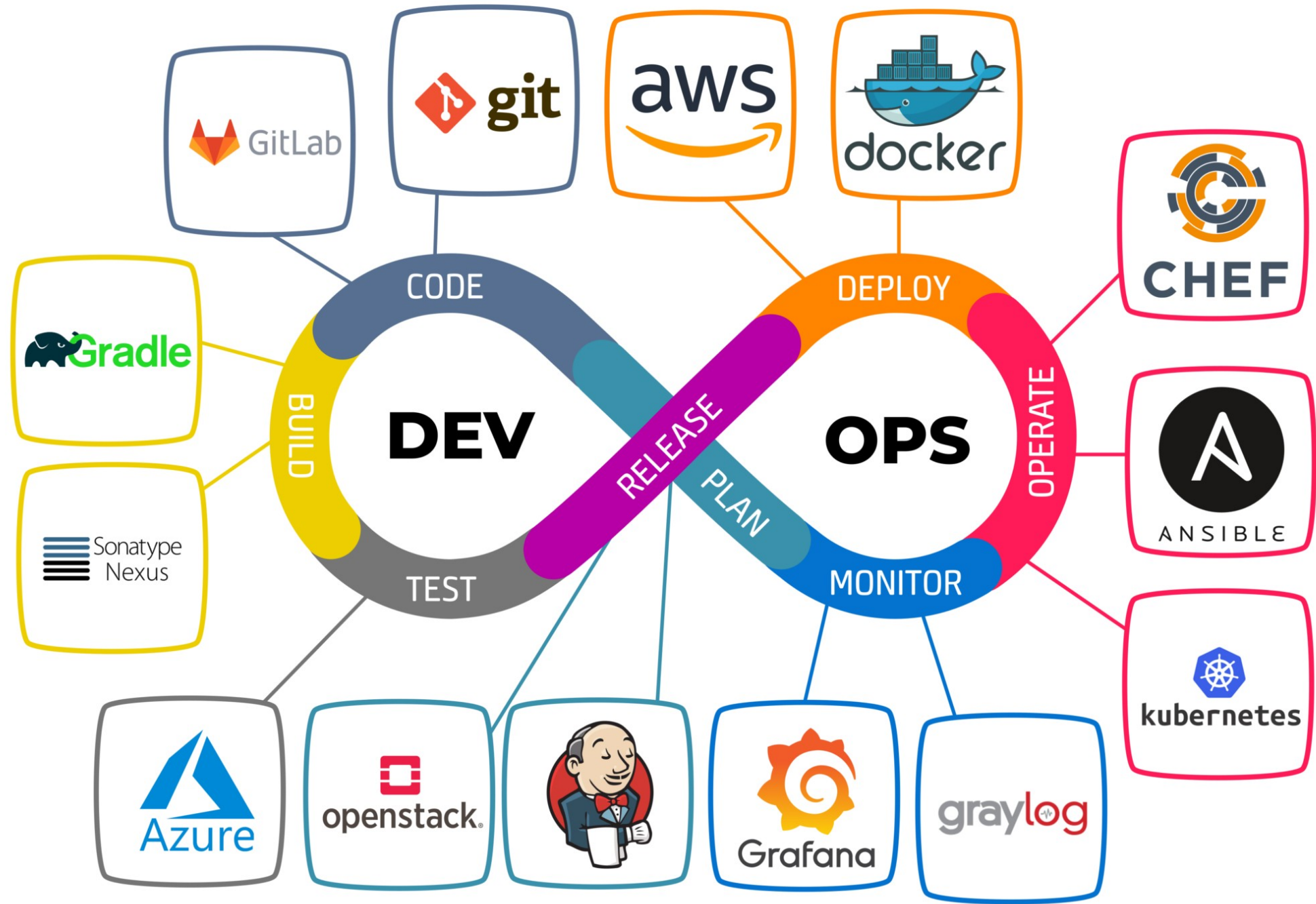
A foundational metaphor in DevOps that describes the traditional, siloed barrier between software development teams (Dev) and IT operations teams (Ops).

It represents the inherent conflict of interest, lack of communication, and friction that occurs when these two departments work in isolation.



The Consequences of the Wall

- **Slow Deployments:** Features take weeks or months to get from development to production.
- **High Failure Rates:** Because Dev doesn't understand the production environment and Ops doesn't fully understand the code, deployments often break.
- **Toxic Culture:** Blame, frustration, and lack of trust build up between the two departments.

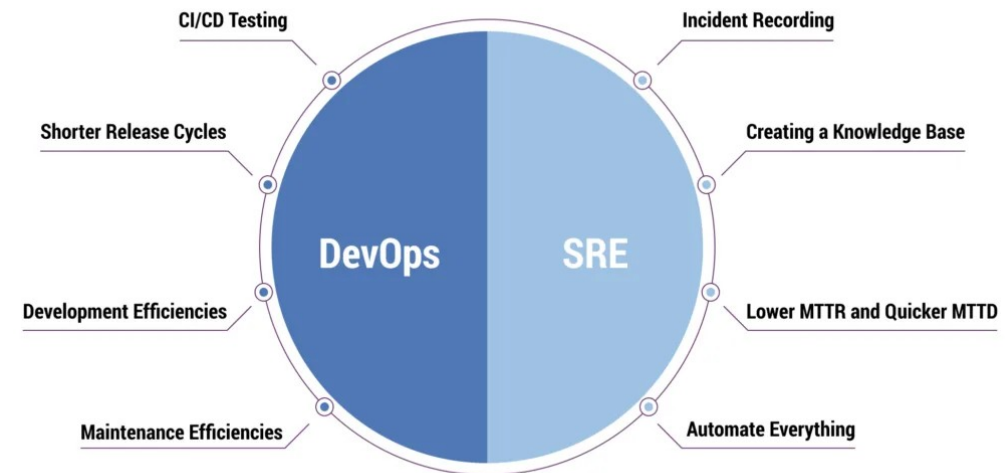


What is Site Reliability Engineering

- SRE treats operations as a software engineering problem
 - “Class SRE implements interface DevOps”
- Solves the “Wall of Confusion” Relevant aspects:
 - Isolation/Visibility: limit what can be seen by tenants
 - Resource control: limit resource consumption
 - Portability: reconstruct the same environment in multiple hosts
- Bridges the gap with:
 - Shared Ownership
 - Failure acceptance
 - Capped Operational Work

Shared Responsibility: Breaking the Silos

- **The Traditional Divide:** Developers want velocity (shipping new features); Operators want stability (keeping systems up).
- **The SRE Shift:** Reliability is no longer just an "Ops problem." It is a shared goal.
- **A Common Language:** Service Level Objectives (SLOs) align both teams on what "healthy" looks like.
- **Unified Ownership:** Developers share the pager; SREs help build the deployment pipelines.



Failure Acceptance: 100% is the Wrong Target

- **Failure is Inevitable:** Hardware degrades, networks drop, software has bugs, threat actors attack, and humans make mistakes.
- **The Cost of 100%:** Attempting perfect reliability stifles innovation and is cost-prohibitive.
 - 99.9% reliability can cost 10x more than 99%
- **Managing Risk (Error Budgets):** A mathematical allowance for acceptable failure over a period of time.
 - Errors are always present, so there is risk to be minimized everywhere
- **Blameless Culture and Postmortens:** When failures happen, fix the system, don't punish the person.
 - Punishing persons doesn't bring back systems, and will result in toxic culture.

Capped Operational Work: Taming the "Toil"

- **Toil:** Work that is manual, repetitive, automatable, tactical, and scales linearly with service growth.
- **The 50% Rule:** SREs must spend at least 50% of their time on engineering work, not just operational tasks.
- Software engineering should eliminate manual system administration, allowing auto-scaling
 - Ops are developers of framework tools and automations

SRE Principles

- **Reliability is King:** SRE considers achieving stability and reliability in a system as the most critical aspect.
- **Users Define Reliability:** SRE considers that monitoring tools do not reflect user experience
 - Components of a user journey must be available when users request it
 - Allows for continuous maintenance tasks
- **Well engineering software/ops will not be enough**
 - Business alignment and effectiveness is required for extra reliability

Error Budgets

Critical User Journey (CUJ)

An important workflow that makes the user happy ... or unhappy

SLI

service level **indicator**: a well-defined measure of 'good enough'

SLO

service level **objective**: a top-line target for fraction of good interactions

SLA

service level **agreement**: an SLO with -generally- financial-consequences



Error Budget

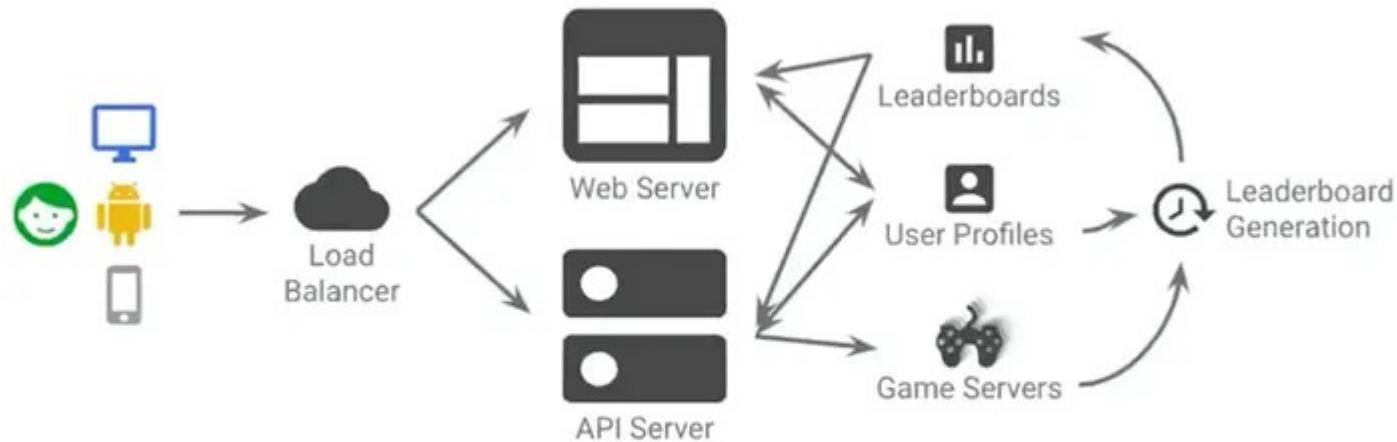
Amount of **unreliability** allowed to operate within SLO



© 2021 Google LLC. All rights reserved.

Critical User Journeys

- If users define reliability, user interactions need to be known, understood and measured.
- User Journey happens when users interact with system to reach a goal
- Some are Critical for business



Service Level Indicators

- An SLI is a metric created to measure reliability, derived and adapted from the monitoring metrics we use.
- A good SLI, can serve as the basis for determining whether users are happy or unhappy with the service.
 - A CUJ shall have up to 5 SLIs
 - SLIs need to have good signal to noise ratio

$$\text{SLI} : \left(\frac{\text{good events}}{\text{valid events}} \right) \times 100\%$$

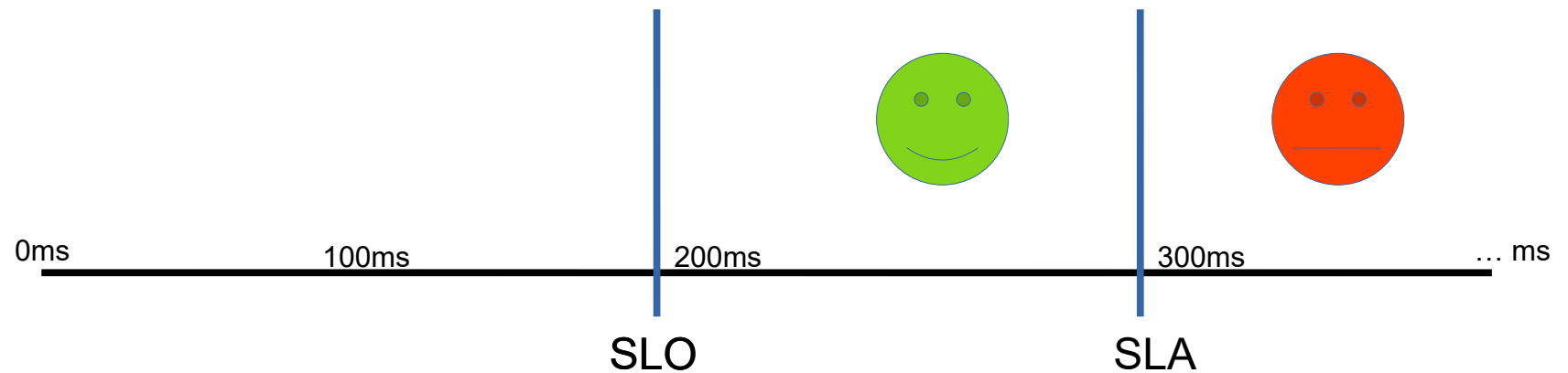
Events with non interaction errors or informative responses are not valid (HTTP 4XX, 100).

Service Level Objectives

- A SLO is an internal objective for operations
 - Should be ambitious
 - Should be achievable
- A SLO should be scoped to the product and user expectations
- SLOs help defining a budget and scoping tasks
 - If the SLO is constantly met, either it is adjusted, or no further optimizations are required

Service Level Agreements

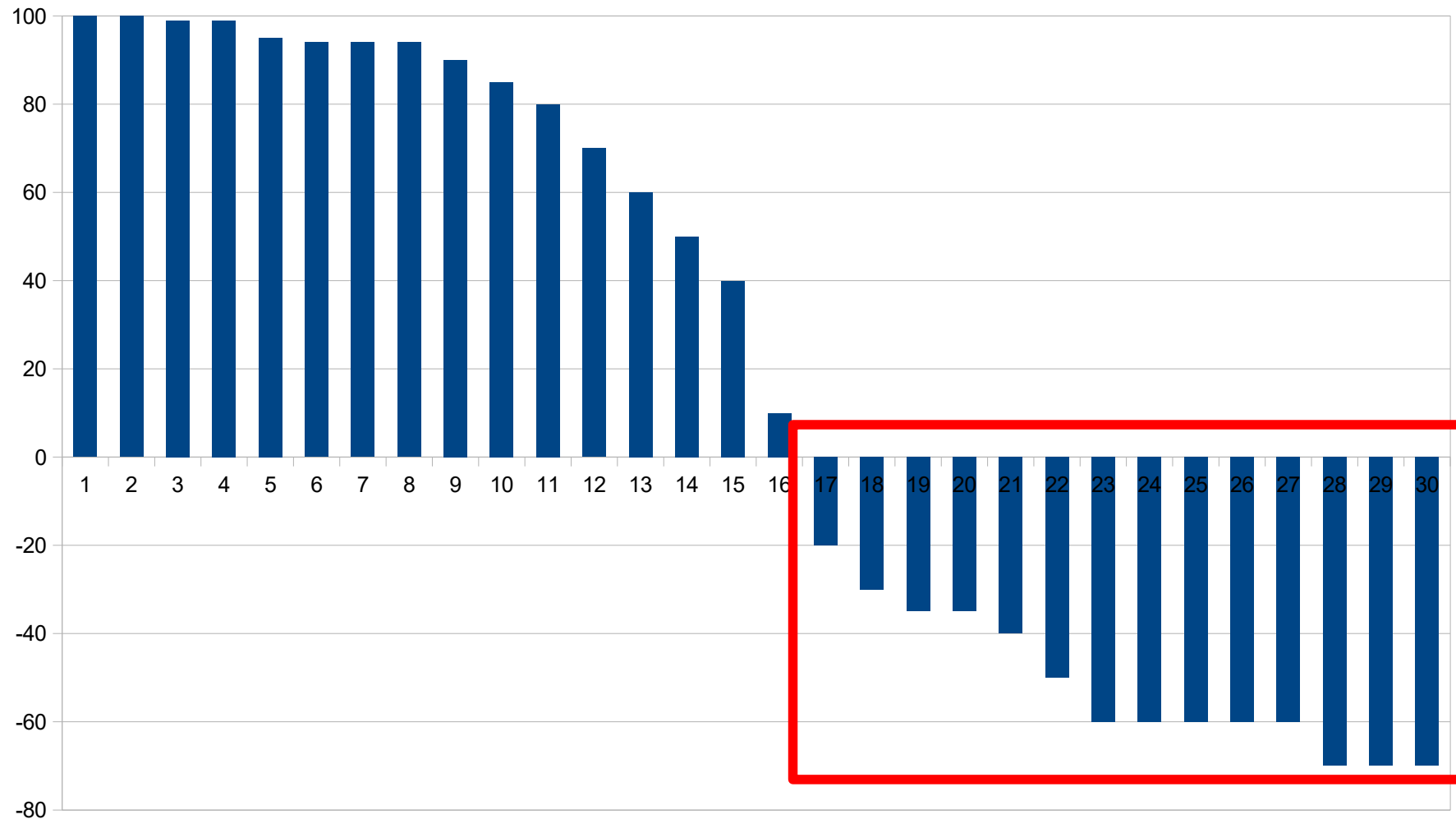
- A SLA is an agreement with stakeholders
 - Define a value above SLOs
 - Violation of SLAs may have financial impact
 - Like SLOs are an agreement with the organization



Error Budgets

- The Goal: 99.9% uptime for the checkout service over a 30-day window.
 - 30 days = 43,200 minutes.
 - Accepted by all stakeholders
- Error Budget: 0.1% of 43,200 = **43.2 minutes** of allowed downtime.
- The Policy:
 - If Budget > 0 : Ship new features!
 - If Budget ≤ 0 : Feature freeze. 100% focus on reliability
 - Error Burndown = when budget goes below 0
- When the window resets, new features can be added.

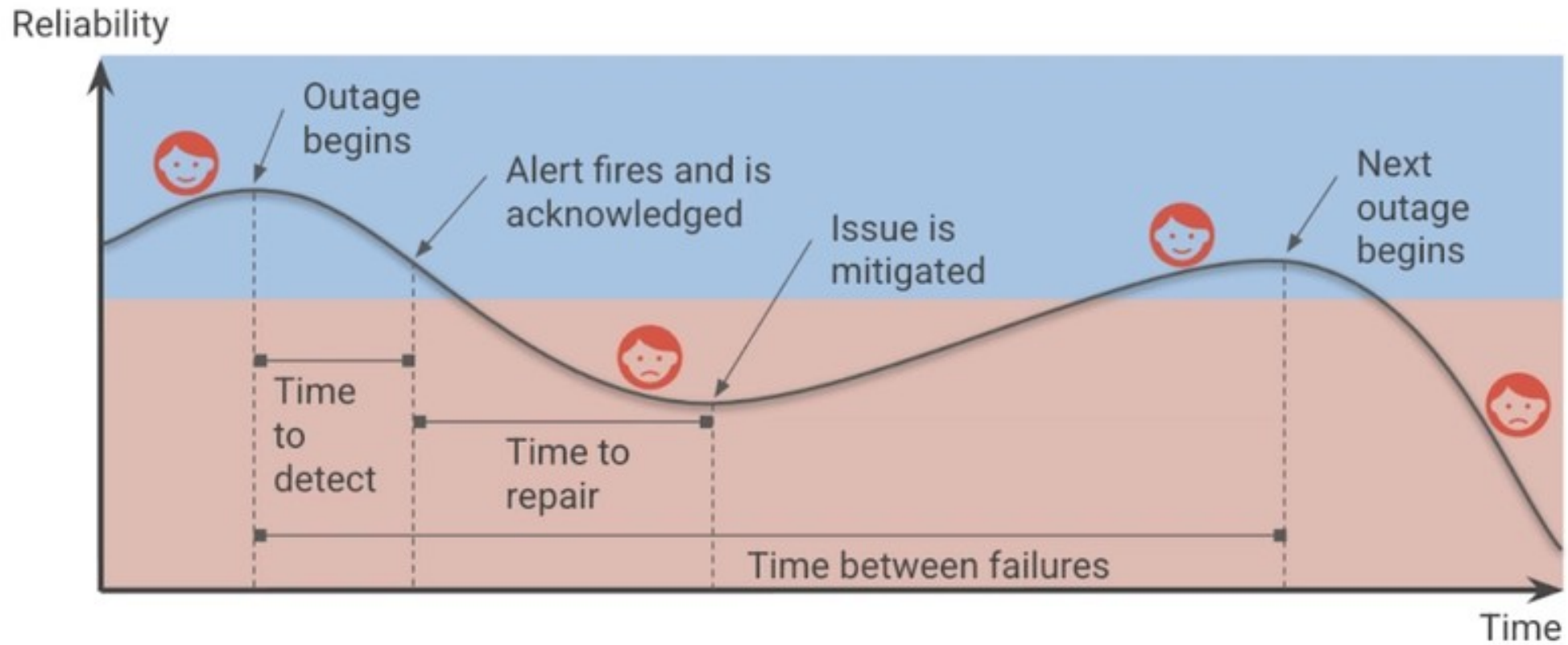
Error Budgets (EB) Burndown



Risk Estimation

- **ETTD** - Estimated Time To Detection – how long it would take to detect and notify a human (or robot) that the incident has occurred;
 - Also MTTD (Mean Time to Detect)
- **ETTR** - Estimated Time To Resolution – how long it would take to fix the incident once the human (or robot) has been notified;
 - Also MTTR (Mean Time to Repair).
- **ETTF** - Estimated Time To Failure – estimated frequency between instances of this incident;
 - Also MTBF (Mean Time Between Failure).
- % of Users Affected – Percentage of users was affected by the failure

Risk Estimation



Risk Estimation

- For every identified risk, ETTD, ETTR, %impact and ETTF is observed
- Data from actual incidents can be used to define probability

Risk	(in minutes)		(in days)		incidents/year	bad mins/year
	ETTD	ETTR	% Impact	ETTF		
Disaster recovery drill	5	60	100%	365	1.00	65
Update major code version	5	120	5%	1095	0.33	2
Code deployment misconfiguration	30	180	40%	607	0.60	51
Aggressive database query causing system to down	30	30	30%	60	6.09	110
Software defects in the code	60	240	20%	219	1.67	100
Breakdown of caching service	30	240	30%	365	1.00	81
Outage in the availability zone of the data center	5	60	33%	1095	0.33	7
Unintended data loss or corruption	240	1,440	5%	1095	0.33	28
Malicious security breach/attack	80	1,440	50%	487	0.75	570
High volume of traffic	240	60	25%	1095	0.33	25
Breakdown in the message queue system	5	300	25%	548	0.67	51
Disk failure	30	60	1%	1460	0.25	0
Third-party dependencies failure	240	900	20%	548	0.67	152

<https://dev.to/coingecko/error-budgets-in-practice-a-data-driven-approach-to-risk-and-release-management-2m9b>

Risk Estimation

- Organization defines a Error Budget, which will affect the priorities of SRE staff
- If it exceeds the expected EB, it becomes high priority

	1242			
Threshold of unacceptability for an individual risk (% of error budget)	25.00%			
Target Availability	99.95%	99.90%	99.50%	
Budget (m/yr)	262.98	525.96	2629.8	
Accepted (m/yr)	0	0	0	
Unallocated Budget (m/yr)	262.98	525.96	2629.8	
Too Big Threshold (m/yr)	65.745	131.49	657.45	
Computed Stack Rank of Risks	bad mins/year	accept risk?	accept risk?	accept risk?
Malicious security breach/attack	570			
Third-party dependencies failure	152			
Aggressive database query causing system to down	110			
Software defects in the code	100			
Breakdown of caching service	81			
Disaster recovery drill	65			
Breakdown in the message queue system	51			
Code deployment misconfiguration	51			
Unintended data loss or corruption	28			
High volume of traffic	25			
Outage in the availability zone of the data center	7			
Update major code version	2			
Disk failure	0			

<https://dev.to/coingecko/error-budgets-in-practice-a-data-driven-approach-to-risk-and-release-management-2m9b>

Culture: Blameless Post-Mortems

- **Systems will fail:** Pods will OOMKill (Out of Memory). Network policies will misconfigure.

The Blameless Philosophy

- When the app goes down, we don't fire the engineer who pushed the bad YAML.
 - We ask: "Why did our CI/CD pipeline allow bad YAML to be applied?" or "Why didn't our K8s test environment catch this before production?"
-
- **Actionable Takeaways:** Every outage must result in an action item
 - writing a new automated test, adding a resource limit to a K8s manifest.

How Kubernetes Fits

- Kubernetes is highly structured, and managed with YAML Files
 - Can be stored in code repository
 - Can be reviewed, audited and changed, with history kept
- It natively supports mechanisms and checks enabling SRE:
 - **Self-Healing:** If a node dies, K8s spins up a new pod. This eliminates the toil of manually restarting servers at 3 AM.
 - **Health Checks:** Liveness and Readiness probes are essentially automated SRE responders, ensuring traffic only goes to healthy pods.
 - Running pods != Healthy pods!
 - **Autoscaling (HPA):** HPA handles traffic spikes automatically, protecting the SLO without human intervention.
 - **Resource Definitions:** Each resource can validate their content (e.g. secrets), preventing errors.